

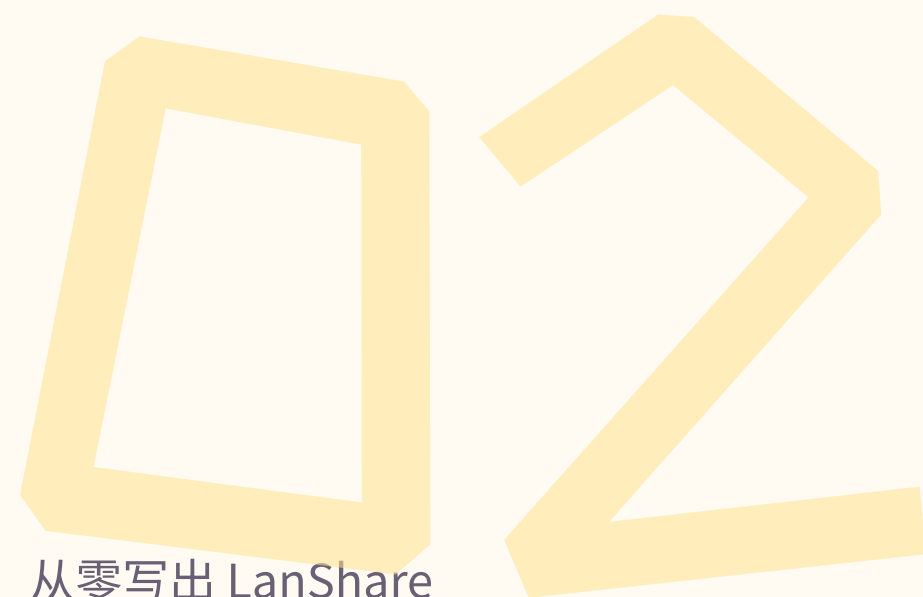


第 2 集 · 约 18 分钟

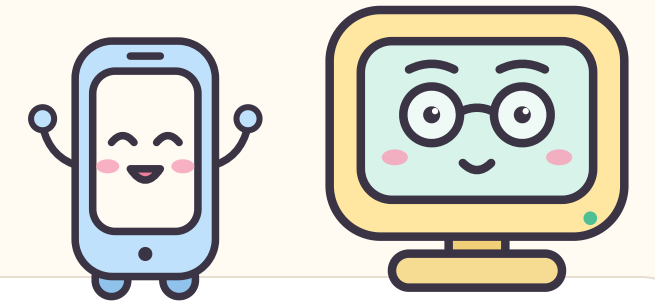
搭骨架：一个工作区，三个 crate

第一行代码之前，先把房子的框架立起来

配套阅读：书的第 1 章



从零写出 LanShare



这一集你会学到

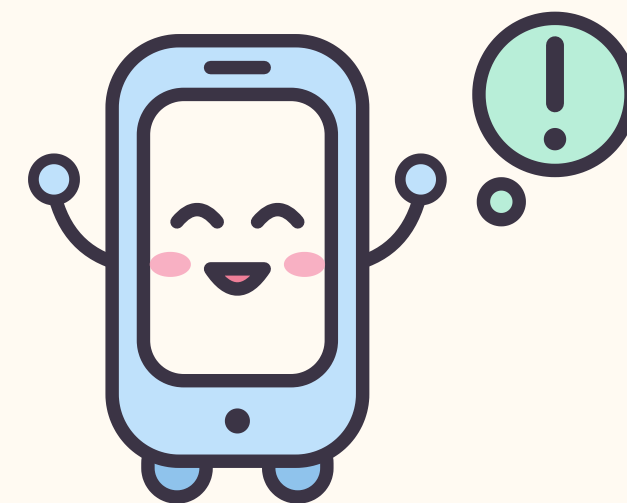
1. 为什么 LanShare 要拆成三个 crate
2. 工作区 Cargo.toml：共享版本、edition 和编译配置
3. 依赖怎么写：版本号、features、路径依赖
4. 文档注释三兄弟：`//!`、`///`、`//`



先想一个问题

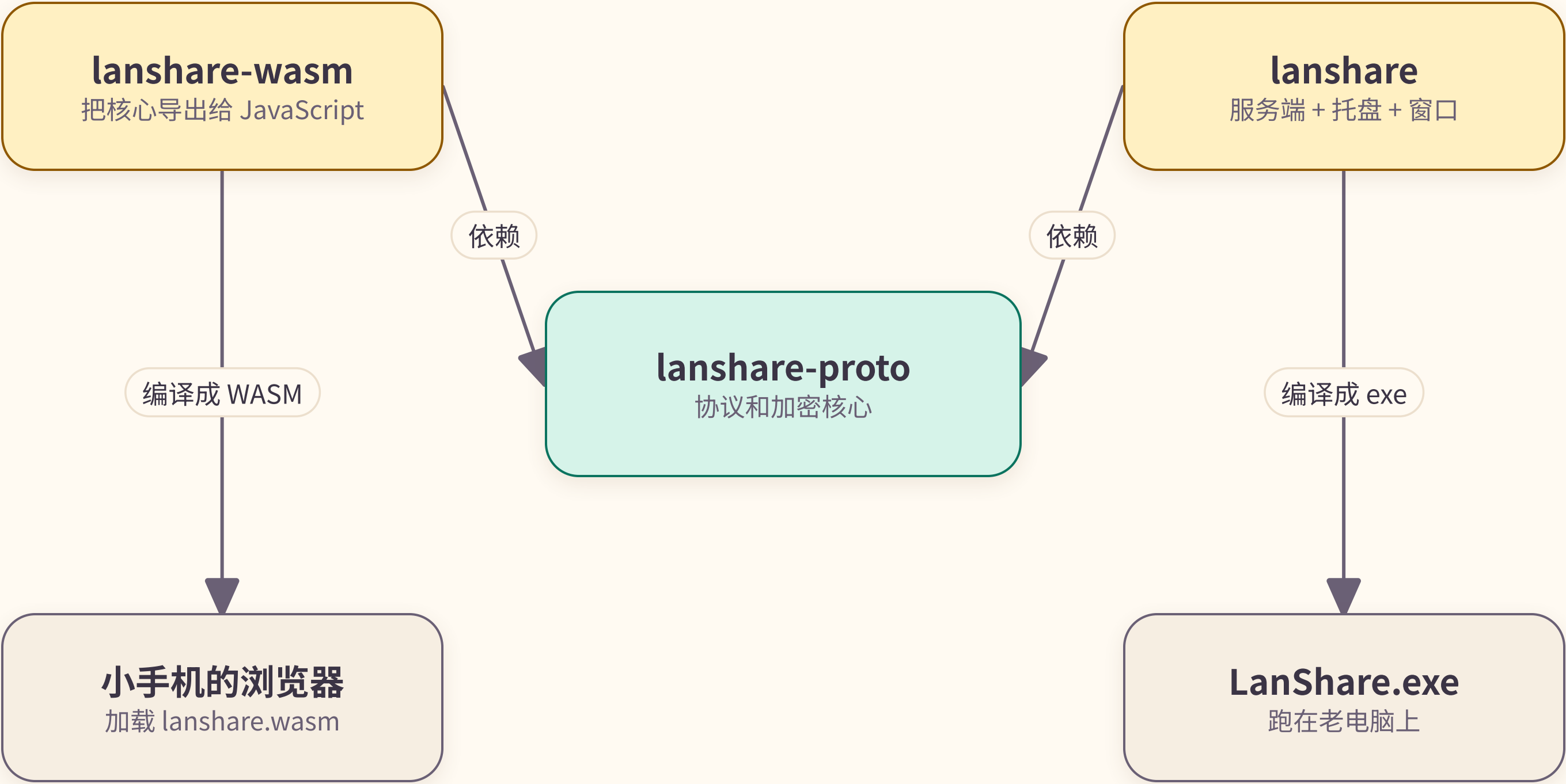
一个小小的局域网快传工具，为什么要拆成三个 crate?

因为同一份加密代码，要在两个完全不同的地方运行：老电脑上的服务端，和小手机的浏览器里。





三个 crate，各管一摊





依赖清单：Python 和 Rust

Python v1 • requirements.txt

```
1 qrcode==8.2
2 pystray==0.19.5
3 Pillow==10.4.0
```

Rust v2 • Cargo.toml (节选)

```
1 [dependencies]
2 axum = "0.8"
3 lanshare-proto = { path = "../lanshare-proto" }
4 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
```



建文件夹，生成三个 crate

```
Windows PowerShell

PS D:\code> cargo --version
cargo 1.96.0 (30a34c682 2026-05-25)

PS D:\code> git init lanshare
Initialized empty Git repository in D:/code/lanshare/.git/

PS D:\code> cd lanshare

PS D:\code\lanshare> cargo new --lib crates/lanshare-proto
  Creating library `lanshare-proto` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

PS D:\code\lanshare> cargo new --lib crates/lanshare-wasm
  Creating library `lanshare-wasm` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

PS D:\code\lanshare> cargo new crates/lanshare
  Creating binary (application) `lanshare` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
```



根目录的 Cargo.toml



Cargo.toml

第 3-22 行 · 共 22 行

```
3 members = ["crates/lanshare-proto", "crates/lanshare-wasm", "crates/lanshare"]
4
5 [workspace.package]
6 version = "2.0.0"
7 edition = "2024"
8 publish = false
9
10 [profile.release]
11 opt-level = 3
12 lto = true
13 codegen-units = 1
14 strip = true
15
16 # 开发构建里，依赖库（加密、HTTP）也开优化：自己的代码照样能调试，测试不用等慢吞吞的未优化加密
17 [profile.dev.package."*"]
18 opt-level = 3
19
20 # 加密库是泛型代码，在调用它的 crate 里才实例化、按调用方的优化级别编译：只优化依赖不够，协议 crate 也要优化
21 [profile.dev.package.lanshare-proto]
22 opt-level = 3
```



发布版的四个开关

配置	做了什么	代价
opt-level = 3	最高优化级别（发布版默认就是 3，这里写明白）	编译更慢
lto = true	链接时跨 crate 优化：能把依赖库的代码一起内联、删掉没用到的部分	编译明显更慢
codegen-units = 1	不把一个 crate 切成多块并行编译，优化器能看到全貌	编译更慢
strip = true	去掉调试符号，exe 更小	崩溃时的栈信息更少



协议核心的 Cargo.toml



Cargo.toml

crates/lanshare-proto/

第 1-14 行 • 共 14 行

```
1 [package]
2 name = "lanshare-proto"
3 description = "LanShare 协议与加密核心：服务端（原生）和浏览器（WASM）共用"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [dependencies]
9 chacha20poly1305 = { version = "0.10", default-features = false, features = ["alloc"] }
10 hkdf = "0.12"
11 hmac = "0.12"
12 rand_core = "0.6"
13 sha2 = "0.10"
14 spake2 = { version = "0.4", default-features = false }
```



协议核心的密码学工具箱

库	用来做什么	后面哪一集用到
chacha20poly1305	加密并防篡改：每条消息一把锁，一个字节被改都能发现	协议①加密信道
hkdf	从一把主密钥派生出多把用途不同的钥匙	协议②握手
hmac	证明“我知道这把钥匙”，但不把钥匙发出去	协议②握手
sha2	哈希函数，HKDF 和 HMAC 的底层	协议②握手
spake2	只凭一个 6 位口令也能安全地协商出密钥	协议②握手
rand_core	随机数的统一接口：电脑和浏览器各自接上自己的随机源	协议②、WASM





WASM 导出层的 Cargo.toml

● ● ●

Cargo.toml

crates/lanshare-wasm/

第 1-12 行 · 共 12 行

```
1 [package]
2 name = "lanshare-wasm"
3 description = "把 lanshare-proto 导出给浏览器（裸 C ABI，不依赖 wasm-bindgen）"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [lib]
9 crate-type = ["cdylib", "rlib"]
10
11 [dependencies]
12 lanshare-proto = { path = "../lanshare-proto" }
```



主程序的 Cargo.toml

● ● ●

Cargo.toml crates/lanshare/

第 1-13 行 • 共 13 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 lanshare-proto = { path = "../lanshare-proto" }
```



文档注释：Python 和 Rust

Python

```
1 """模块说明：写在文件最开头的字符串。"""
2
3 PROTOCOL = "lanshare/2"
4 """常量说明（只是约定，工具不一定认）"""
5
6 # 普通注释，给读代码的人看
```

Rust

```
1 //! 模块说明：写在文件最开头，说明“这个文件”
2
3 /// 常量说明：写在条目正上方，说明“下面这个东西”
4 pub const PROTOCOL: &str = "lanshare/2";
5
6 // 普通注释，不进文档
```



协议核心的入口 lib.rs

lib.rs crates/lanshare-proto/src/第 1-7 行 • 共 7 行

```
1  //! LanShare 协议与加密核心：服务端（原生）和浏览器（WASM）共用同一份代码。
2  //!
3  //! 协议细节见 `docs/superpowers/specs/2026-10-02-lanshare-v2-rust-design.md` §3-§4。
4  //! 这里的每一个常量和标签字符串都是协议的一部分，改动会让两端互不相认。
5
6  /// 协议版本，出现在 HKDF 的 info 和 AEAD 的 aad 里。
7  pub const PROTOCOL: &str = "lanshare/2";
```



WASM 导出层的 lib.rs

lib.rs crates/lanshare-wasm/src/ 第 1-9 行 • 共 9 行

```
1  //! 把 `lanshare-proto` 导出给浏览器：裸 C ABI，不依赖 wasm-bindgen。
2  //!
3  //! 调用约定（由 `web/proto.js` 遵守）：
4  //! - 所有指针都指向本模块线性内存里、由 JS 通过 [`ls_alloc`] 申请的缓冲区；
5  //! - 输入缓冲区长度由参数给出，输出缓冲区的长度由函数文档规定，JS 必须先申请够；
6  //! - 64 位计数器拆成高低两个 u32 传，兼容不支持 BigInt↔i64 的老浏览器。
7  //!
8  //! 浏览器里的 WASM 拿不到操作系统随机源，随机数通过导入函数 `env.ls_random`
9  //! 由 JS 的 `crypto.getRandomValues` 提供（它在 http 页面里也可用）。
```



主程序的 lib.rs

lib.rs crates/lanshare/src/第 1-1 行 · 共 1 行

1

//! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。





先放一个临时的 main.rs

● ● ●

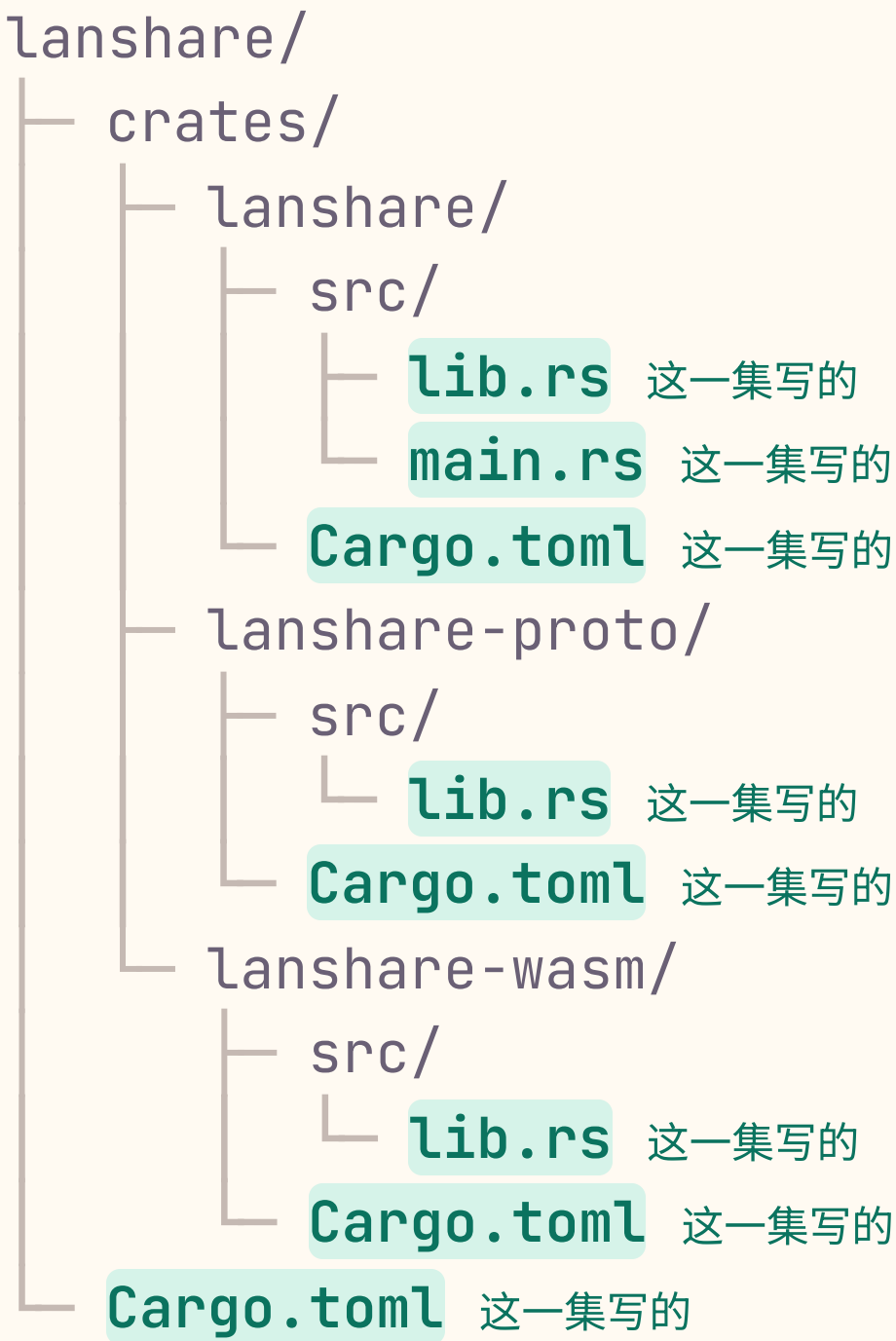
main.rs crates/lanshare/src/ 临时

第 1-3 行 · 共 3 行

```
1 fn main() {
2     println!("LanShare 骨架跑起来了，协议版本: {}", lanshare_proto::PROTOCOL);
3 }
```



骨架搭好了





编译、运行

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Updating crates.io index
  Locking 33 packages to latest Rust 1.96.0 compatible versions
  Adding chacha20poly1305 v0.10.1 (available: v0.11.0)
  Adding generic-array v0.14.7 (available: v0.14.9)
  Adding hkdf v0.12.4 (available: v0.13.0)
  Adding hmac v0.12.1 (available: v0.13.0)
  Adding rand_core v0.6.4 (available: v0.10.1)
  Adding sha2 v0.10.9 (available: v0.11.0)
  Compiling version_check v0.9.5
  ..... (省略 30 行, 都是 Compiling 依赖库)
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 14.03s

PS D:\code\lanshare> cargo run -p lanshare
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
  Running `target\debug\LanShare.exe`
LanShare 骨架跑起来了, 协议版本: lanshare/2
```





选择题

根目录 Cargo.toml 写的是 `version = "2.0.0"`，lanshare-proto 的 Cargo.toml 里 `version.workspace = true`，它的版本是多少？

A 2.0.0

B 0.1.0

C 没有版本，编译报错



选择题

根目录 **Cargo.toml** 写的是 **version = "2.0.0"**，**lanshare-proto** 的 **Cargo.toml** 里 **version.workspace = true**，它的版本是多少？

A 2.0.0

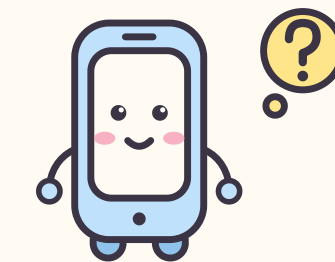
workspace = true 的意思就是照抄工作区 [workspace.package] 里的值。

B 0.1.0

这是 cargo new 生成的默认版本，我们已经把它换掉了。

C 没有版本，编译报错

继承来的版本和直接写一样有效。



选择题

开发版里，只给依赖库开优化（`package."*"`），为什么加密还是很慢？

- A 因为星号不包括密码学库
- B 因为泛型代码在调用方 crate 里才实例化，按调用方的优化级别编译
- C 因为开发版不能开优化



选择题

开发版里，只给依赖库开优化（`package."*"`），为什么加密还是很慢？

A 因为星号不包括密码学库

星号包括所有依赖库，chacha20poly1305 也在里面。

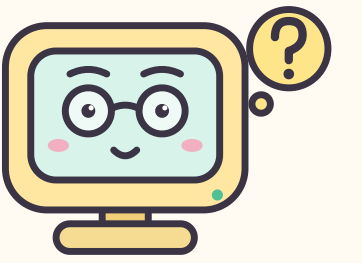
C 因为开发版不能开优化

可以，`profile.dev` 里每个包都能单独设置 `opt-level`。

B 因为泛型代码在调用方 crate 里才实例化，按调用方的优化级别编译

加密库的核心是泛型，真正生成机器码是在 `lanshare-proto` 里，所以 `lanshare-proto` 也要开优化。

想一想



lanshare 和 lanshare-wasm 都依赖 lanshare-proto。执行一次 cargo build，lanshare-proto 会被编译几次？如果再编译到 WASM 目标呢？



想一想

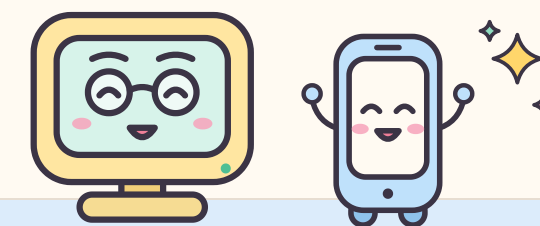
lanshare 和 lanshare-wasm 都依赖 lanshare-proto。执行一次 cargo build，lanshare-proto 会被编译几次？如果再编译到 WASM 目标呢？

一次 cargo build 只编译一次：工作区共用一个 target 目录，同一个目标平台上同一个依赖只编译一份，两个 crate 共用。编译到 WASM 时，目标平台变了，机器码完全不同，所以会在 target 下的 wasm32-unknown-unknown 目录里再编译一份。



这一集写了什么

- ✓ 三个 crate：协议核心、WASM 导出层、主程序；核心只写一份，两端共用
- ✓ 根 Cargo.toml 管工作区：成员、共享版本和 edition、发布版和开发版的编译配置
- ✓ 依赖写“能接受的版本”，Cargo.lock 记“实际用的版本”；路径依赖把 crate 连起来
- ✓ 泛型在调用方实例化，所以开发版也要给 lanshare-proto 开优化
- ✓ `#![` 说明文件，`///` 说明下面的条目，`//` 是普通注释



下一集

全课完结，感谢收看